
Hardware/Software Design Description

For

CourtSweeper: A Vision-Based Tennis Ball Retriever with Dual-Roller Intake Mechanism

Version 2.0 approved

Group 13

Date 2026-05-25

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

Revision History

Name	Date	Reason For Changes	Version
Yilin Zhang	2026-03-17	Initial document creation and LaTeX template configuration.	0.1
Yingrui Sun	2026-03-18	Drafted initial hardware specifications, including dual-roller mechanism and power system.	0.2
Shijia Luo	2026-03-18	Drafted software architecture, including machine vision algorithms and communication protocols.	0.3
Yuwei Zhao	2026-03-23	Comprehensive review and optimization. Finalized system design and approved Baseline 1.0.	1.0

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

Table of Contents

1	Introduction	1
1.1	Purpose	1
1.2	Scope	1
1.3	Definitions, Acronyms and Abbreviations	2
1.4	References	3
1.5	Organization of Document	3
2	Hardware Design	5
2.1	Camera Setup	5
2.2	DJI RoboMaster EP Chassis	6
2.3	Custom Intake Mechanism	7
2.4	LiDAR Sensor Setup	7
2.5	Jetson Orin NX Setup	8
2.6	Power Management System	9
3	Software Design: Autonomous Navigation and System Architecture	11
3.1	Architecture Overview	11
3.2	Chassis Navigation & DJI SDK Integration	12
3.3	SLAM-Based Mapping Pipeline	13
3.3.1	RoboMaster Chassis Bridge (map_generation.py)	13
3.3.2	LiDAR Integration & Filtering	13
3.3.3	SLAM Solver (slam_toolbox)	14
3.4	Nav2-Based Autonomous Navigation	14
3.4.1	Localization and Planning	14
3.4.2	RoboMaster Navigation Bridge (map_navigation.py)	15
3.4.3	Waypoint-Based Coverage Strategy	15
3.5	Video I/O & Pre-processing	15
3.6	Video Processing (YOLO & OpenCV)	15
3.7	Hierarchical Finite State Machine Design	16
3.7.1	iOS Interrupt Control Layer (Upper FSM)	16
3.7.2	Autonomous Workflow Layer (Lower FSM)	17
3.8	Class Design	18
4	Software Design: Perception and Reactive Control	20
4.1	Computer Vision & Object Detection Module	20
4.1.1	Module Architecture	20
4.1.2	Dataset Collection	20
4.1.3	Model Training	21
4.1.4	Cross-Platform Model Deployment	22
4.1.5	Real-Time Object Detection Pipeline	23
4.1.6	Visual Overlay & Debug Information	25
4.1.7	Inter-Module Data Exchange	26

4.1.8	Summary	26
4.2	Intake Motor Control Module	26
4.2.1	Hardware Driver Architecture	27
4.2.2	Low-Level Firmware Design	27
4.2.3	High-Level Control Interface	28
4.2.4	FSM Integration & Intake Trigger Logic	29
4.3	Proximity-Based Chassis Navigation Module	31
4.3.1	Distance Sensing Submodule	31
4.3.2	PID Controller Configuration	32
4.3.3	Chasing Sequence State Tracking	33
4.3.4	Multi-Stage Approach & Ingestion Logic	33
4.3.5	Chassis Drive Primitives	35
4.3.6	Idle & Search Behavior	36
5	User Interface Design	37
5.1	Main Dashboard & Video HUD	37
5.2	Teleoperation & Control Panel	38
5.3	Telemetry & Log Console	38
6	Theoretical Manual	40
6.1	Perception Models	40
6.2	Mecanum Wheel Kinematics	40
6.3	PID Control Theory	40
6.4	Navigation and Planning Theory	40
6.5	Finite State Machine	41
6.6	Coordinate Transformations	41
6.7	Communication Protocol Specifications	41
6.8	Motor Drive Theory	41

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

1 INTRODUCTION

1.1 Purpose

This document describes the preliminary design of the CourtSweeper system. Its primary purpose is to provide a comprehensive architectural overview and detailed engineering specifications, serving as the Baseline 1.0 blueprint for the development team. It details the physical mechanical structures, electronic power systems, computer vision algorithms, and the iOS-based user interface to guide the implementation, integration, and testing phases.

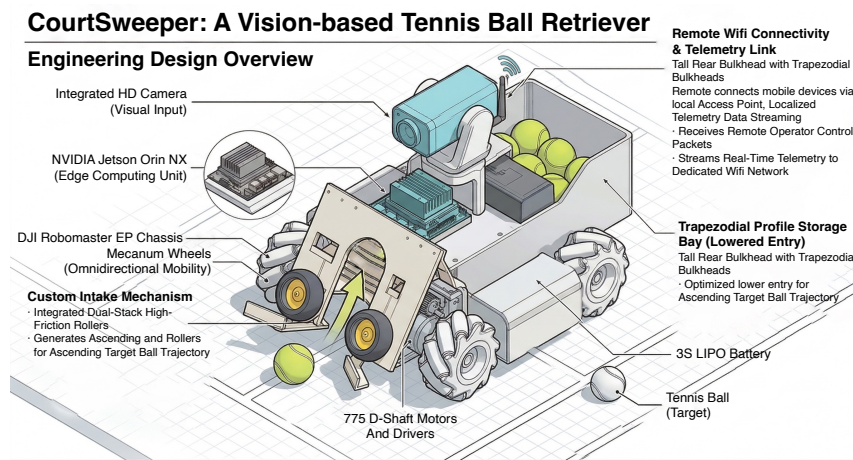


Figure 1: Comprehensive engineering design overview of the CourtSweeper robotic platform.

1.2 Scope

This document describes the design of the CourtSweeper, an intelligent, vision-guided robotic vehicle designed to automate the retrieval of scattered tennis balls in sports facilities. The scope of this system encompasses:

- **Hardware Subsystem:** A hybrid mobile platform utilizing the DJI RoboMaster EP [2] chassis to provide omnidirectional mobility and the primary HD camera feed. It is physically integrated with a custom-built, high-torque dual-roller intake mechanism, which is driven by 775 D-shaft motors and BTS7960 drivers for ball collection, and an NVIDIA Jetson Orin NX [4] acting as the central edge computing unit.
- **Software Subsystem:** An onboard AI pipeline utilizing the RoboMaster-SDK-Ultra [5] (a customized, high-performance fork of the official DJI SDK developed specifically for this project) for vehicle telemetry and camera stream extraction. This is combined with the YOLO [6] object detection model and OpenCV, enabling real-time target recognition, spatial coordinate mapping, and dynamic intake motor control via PWM.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

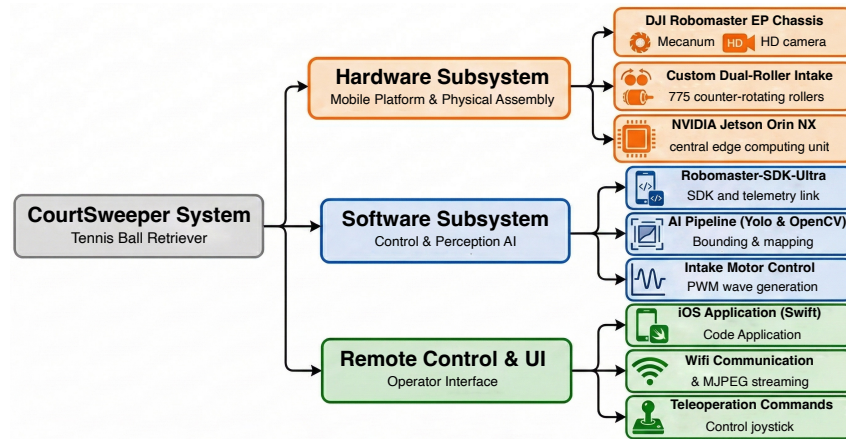


Figure 2: Hierarchical scope tree diagram detailing the overarching architecture of the CourtSweeper system.

- **Remote Control & UI:** A native iOS application [1] developed in Swift, establishing a localized Wi-Fi connection to stream AI-annotated video feeds (via MJPEG) and transmit low-latency teleoperation commands to the Jetson unit.

1.3 Definitions, Acronyms and Abbreviations

Table 1: List of abbreviations and terminology used in the CourtSweeper.

Term	Definition
HSDD	Hardware/Software Design Description.
CS	CourtSweeper, the designated project name for the tennis ball collecting robot.
UI	User Interface, specifically referring to the iOS application in this project.
YOLO	You Only Look Once (specifically yolo26), a SOTA real-time object detection system utilized for high-speed tennis ball recognition.
RoboMaster-SDK-Ultra	A customized, advanced fork of the official DJI SDK (developed by RamessesN), utilized to interface with the RoboMaster EP chassis, execute complex telemetry, and extract raw HD camera feeds.
OpenCV	Open Source Computer Vision Library, utilized for image processing, color space conversion, and bounding box coordinate extraction.
FSM	Finite State Machine, the logic architecture governing the autonomous and manual behavioral states of the robot.
IPC	Inter-Process Communication, a data exchange mechanism between OS processes. The custom SDK circumvents IPC to minimize video latency.
Mecanum Wheels	Omnidirectional wheels on the RoboMaster EP chassis that allow the robot to move in any direction without changing its orientation.
PWM	Pulse Width Modulation, utilized for custom motor speed regulation in the 775 motor-driven intake mechanism.
LiPo	Lithium Polymer, the high-discharge-rate battery chemistry utilized for the heavy-duty 3S actuation power source.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

Table 1 continued

Term	Definition
MJPEG	Motion JPEG, a video compression format used to stream AI-processed (bounding-box annotated) video frames to the iOS client with low latency.
HUD	Heads-Up Display, the primary transparent-style layout utilized in the iOS application for concurrent video and telemetry monitoring.
AP Mode	Access Point Mode, allowing the Jetson Orin NX to act as a localized Wi-Fi router for direct mobile connection in outdoor environments.

1.4 References

- [1] Apple Inc. *Apple Developer Documentation - Network Framework*. 2026. URL: <https://developer.apple.com/documentation>.
- [2] DJI. *RoboMaster EP Programming Guide and SDK Documentation*. 2020. URL: <https://www.dji.com/cn/robomaster-ep>.
- [3] *Dual Roller Mechanism*. 2024. URL: https://www.bilibili.com/video/BV1bgveeoEiX?vd_source=6621406a814999260edb3d6efe8942da.
- [4] Nvidia Inc. *NVIDIA Jetson Orin NX*. 2022. URL: <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/>.
- [5] RamessesN. *Robomaster SDK Ultra*. 2025. URL: <https://github.com/RamessesN/Robomaster-SDK-Ultra>.
- [6] *Ultralytics YOLO26*. 2026. URL: <https://docs.ultralytics.com/models/yolo26/>.

1.5 Organization of Document

To provide a structured and logical flow of the system architecture, this document is organized into the following major sections:

Table 2: Organization of this Hardware/Software Design Description.

Section	Core Content	Description
Section 2 <i>Hardware Design</i>	• DJI RoboMaster EP Chassis • Custom Intake Mechanism • Jetson Orin NX Setup • Power Management System	Provides the physical and electrical blueprint of the robot. It details the integration of the hybrid mobile platform, the custom 775 motor-driven dual-roller mechanism, and the multi-voltage power distribution topology.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

Table 2 continued

Section	Core Content	Description
Section 3 <i>Autonomous Navigation & System Architecture</i>	• ROS2-Based SLAM Mapping • Nav2 Autonomous Navigation • Hierarchical FSM Design • Class Architecture	Describes the overarching system architecture and autonomous navigation pipeline. It details the ROS2 middleware layer bridging DJI hardware with SLAM and Nav2, the hierarchical two-layer FSM governing mission execution, and the modular class design.
Section 4 <i>Perception & Reactive Control</i>	• YOLO Computer Vision Pipeline • Intake Motor Control • Chassis PID Navigation • Post-Ingestion Sector Sweep	Provides the in-depth implementation of the perception and reactive control layer. It covers dataset collection, model training, and real-time YOLO inference; the dual-roller motor firmware and ingestion trigger FSM; the dual-PID chassis controller with blind-zone relay; and the post-ingestion 360-degree sector clearing logic.
Section 5 <i>UI Design</i>	• Main Dashboard & Video HUD • Teleoperation & Control Panel • Telemetry & Log Console	Outlines the Swift-based iOS application architecture, detailing the visual presentation layer, network socket communication, and real-time system monitoring protocols.
Section 6 <i>Theoretical Manual</i>	• Perception Models & Sensor Theory • Mecanum Kinematics & PID Control • SLAM, AMCL & Path Planning • FSM, Coordinate Transforms, & Protocols	Provides a brief overview of theoretical domains (perception, kinematics, PID, navigation, FSM, transforms, protocols, motor theory) and directs the reader to the standalone <i>CourtSweeper Theoretical Manual</i> for complete formal derivations.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

2 HARDWARE DESIGN

The hardware architecture of the CourtSweeper system is a hybrid design, integrating a commercial robotic platform with custom-engineered mechanical and computational subsystems. This section details the physical embodiment, sensor configurations, and electrical topologies required to achieve autonomous tennis ball retrieval.

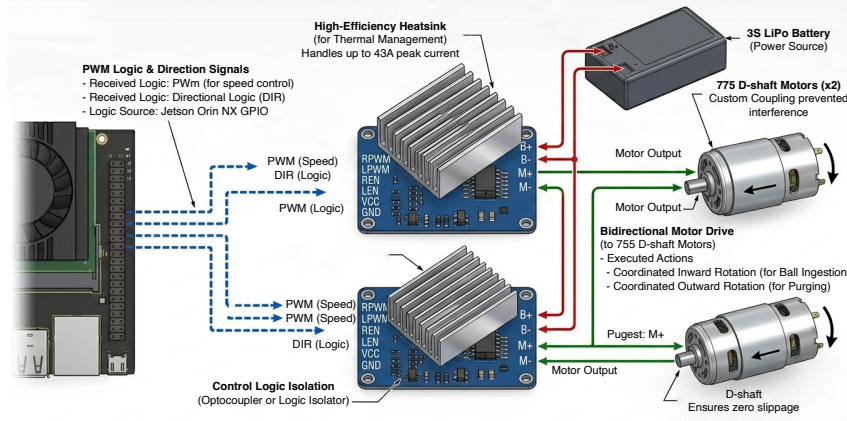


Figure 3: An engineering diagram detailing the BTS7960 high-current drive architecture for the CourtSweeper system, illustrating the connection of an NVIDIA Jetson Orin NX to two BTS7960 motor drivers powered by a 3S LiPo battery to control two 775 D-shaft motors, featuring logic signal connections, power distribution, bidirectional motor drive, and technical callouts for key components like heatsinks and motor features.

2.1 Camera Setup

The primary visual input via the integrated HD camera on the Robomaster EP chassis, which feeds raw video streams to the edge computing unit for target detection. The core hardware parameters of the integrated camera are detailed in Table 3.

Table 3: DJI RoboMaster EP Integrated Camera Specifications

Hardware Parameter	Specification
Sensor Type	CMOS 1/4 , Effective Pixels: 5 MP
Field of View (FOV)	120°
Max Video Resolution	FHD: 1080p@30fps / HD: 720p@30fps
Max Video Bitrate	16 Mbps
Max Still Photo Resolution	2560 × 1440

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026



Figure 4: A physical picture of Robomaster EP Camera Module.

2.2 DJI RoboMaster EP Chassis



Figure 5: A physical picture of Robomaster EP Chassis Module.

The foundation of the CourtSweeper is the DJI RoboMaster EP chassis. Selected for its industrial-grade stability and payload capacity, the chassis is equipped with four Mecanum wheels, granting the robot true omnidirectional mobility. This holonomic drive system is crucial for the precise spatial adjustments required to align the front-facing intake mechanism with scattered tennis balls without needing a turning radius.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

2.3 Custom Intake Mechanism

To achieve reliable and rapid ball collection, a custom dual-roller intake mechanism [3] was designed and mounted at the front of the chassis. The actuation is driven by two high-torque 775 DC motors (D-shaft configuration, rated at 12V and 4600 RPM).

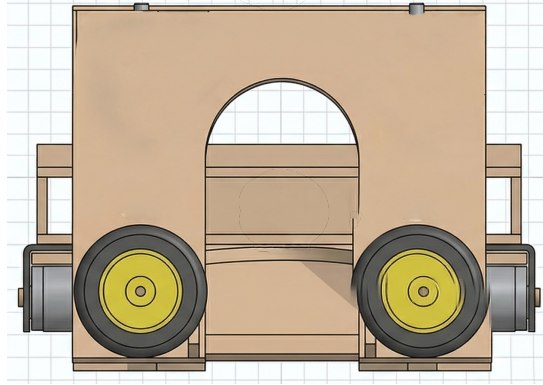


Figure 6: A detailed front-on view of the Courtsweeper dual-roller intake mechanism.

- **Mechanical Coupling:** The motors are mated to 60mm solid rubber wheels featuring high-friction hexagonal grooves via 30mm extended brass couplings. This extended design provides essential clearance to prevent mechanical interference between the rotating wheel hubs and the motor casing, while the D-shaft ensures zero slippage under high torque.
- **Motor Drivers:** Two BTS7960 high-current motor driver modules (capable of handling up to 43A peak current) are utilized to independently control the two motors. They receive Pulse Width Modulation (PWM) and directional logic signals from the central computing unit to execute coordinated inward rotation (for ball ingestion) or outward rotation (for purging).

2.4 LiDAR Sensor Setup

To ensure precise operational boundary gridding and efficient path planning (as outlined in the software architecture Fig.11), the Courtsweeper incorporates a dedicated 2D LiDAR sensor. This sensor provides robust 2D scanning of the court environment and potential obstacles (see §6.1 for the operating principle).

- **Sensor Details:** The selected hardware is the RPLidar A1M8 triangulation-based LiDAR. It is mounted at the rear of the chassis and interfaces with the Jetson Orin NX via a customized USB-serial connection.
- **Specifications:** The A1M8 offers a maximum distance range of up to 12 meters, enabling comprehensive court coverage. Key operational parameters include a sampling frequency of 8000 Hz, a standard rotating frequency of 7 Hz, and an angular resolution of $\leq 1^\circ$.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

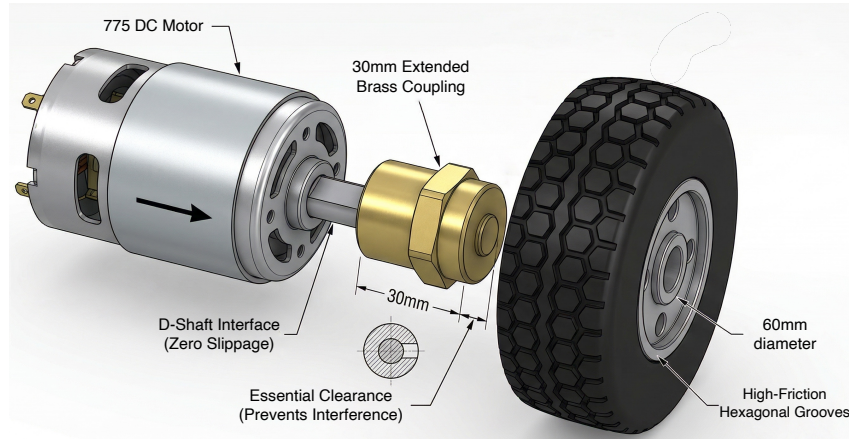


Figure 7: Close-up view of the mechanical coupling. The 30mm extended brass coupling provides critical operational clearance between the 775 motor casing and the high-friction intake roller.



Figure 8: A physical picture of the RPLidar A1M8 LiDAR sensor.

2.5 Jetson Orin NX Setup

The cognitive core of the system is the NVIDIA Jetson Orin NX. Acting as the Edge Computing Unit, it is responsible for handling computationally intensive tasks locally, thereby eliminating cloud-dependency and minimizing latency. The Jetson module runs the Ubuntu operating system and is configured to:

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

1. Execute the YOLO object detection pipeline concurrently with OpenCV operations to process the H.264 video stream captured by the DJI chassis.
2. Run the customized RoboMaster-SDK-Ultra to translate bounding box coordinates into proportional navigation commands.
3. Generate precise PWM signals via its GPIO pins to command the BTS7960 drivers.
4. Operate in Access Point (AP) mode to establish a localized Wi-Fi network for the iOS UI client.



Figure 9: A physical picture of NVIDIA jetson Orin NX.

2.6 Power Management System

Given the diverse voltage requirements and the risk of electromagnetic interference from high-current motors, the CourtSweeper employs an isolated, multi-rail power distribution topology:

- **Mobility Power:** The DJI RoboMaster EP chassis and its onboard camera are powered autonomously by the proprietary DJI Intelligent Battery (3S LiPo, Nominal 10.8V, 2400mAh, 25.92Wh).
- **Actuation Power:** The 775 intake motors and BTS7960 drivers are powered by a dedicated 3S (11.1V) Lithium Polymer (LiPo) battery rated at 5200mAh and 40C discharge. This ensures the motors receive sufficient burst current during ball compression without inducing voltage sags in the logic circuits. A low-voltage buzzer (BX100) is attached to the balance lead for real-time cell monitoring.
- **Logic Power:** The Jetson Orin NX requires a stable 19V input. To prevent logic resets caused by motor back-EMF, it is powered by a galvanically isolated power source.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

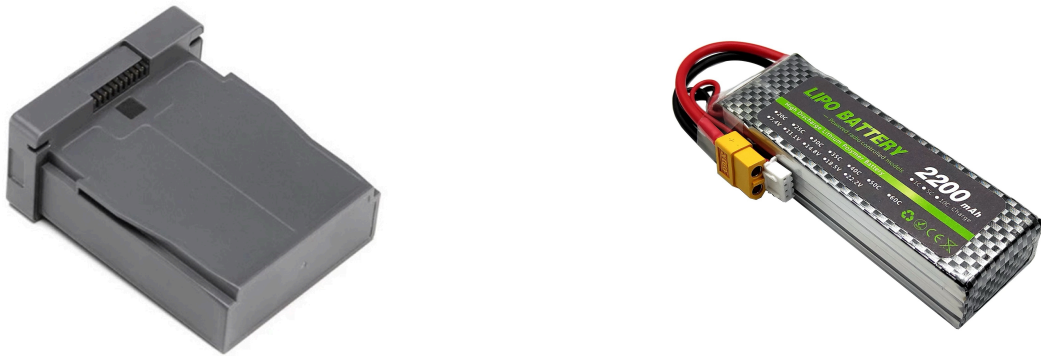


Figure 10: A physical picture of Robomaster EP battery & Motor-driven 3S LiPo battery.

- **Common Grounding:** Crucially, the ground (GND) planes of the Jetson Orin NX and the BTS7960 motor drivers are tied together to ensure reliable logic-level signal transmission.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

3 SOFTWARE DESIGN: AUTONOMOUS NAVIGATION AND SYSTEM ARCHITECTURE

3.1 Architecture Overview

The software subsystem of CourtSweeper is structured as a hierarchical two-layer Finite State Machine (FSM) architecture, as illustrated in Figure 11. All modules run on the NVIDIA Jetson Orin NX edge computing unit, bridging heterogeneous sensory inputs (DJI camera, RPLidar, IR proximity sensor) with mechanical outputs (Mecanum chassis, dual-roller intake) through a ROS2-based middleware layer.

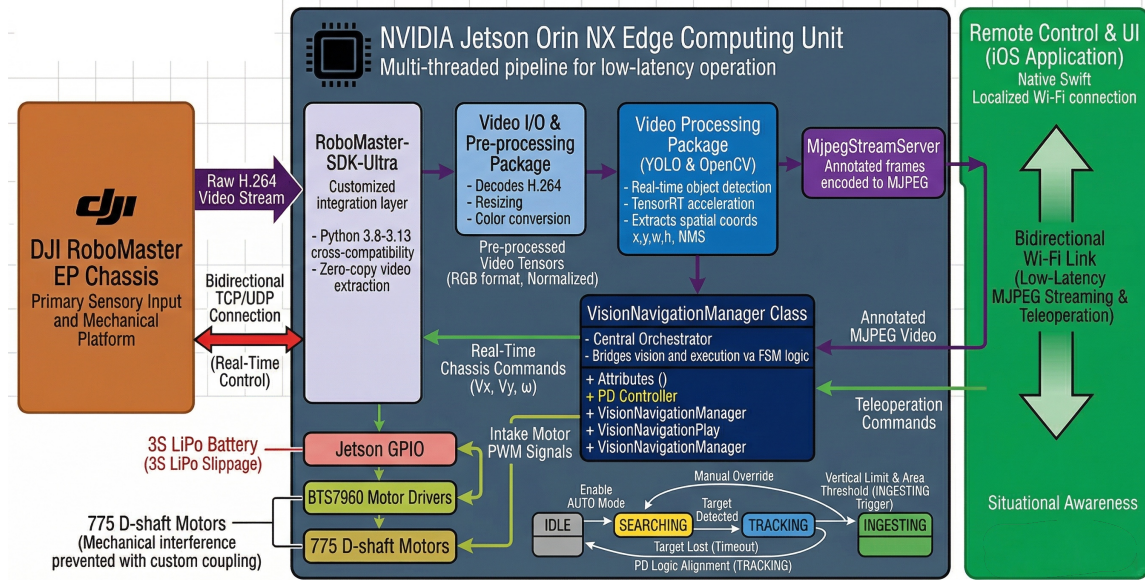


Figure 11: The hierarchical software architecture of CourtSweeper. The upper layer handles iOS interrupt commands, while the lower layer executes the autonomous workflow through three sequential phases.

Every ROS2 bridge node in the system follows a uniform initialization pattern, establishing a 50 Hz control loop timer and subscribing to the `/cmd_vel` topic for navigation command forwarding:

```

1 class RoboMasterBridge(Node):
2     def __init__(self):
3         super().__init__('robomaster_bridge')
4         self.odom_pub = self.create_publisher(Odometry, '/odom', 10)
5         self.camera_pub = self.create_publisher(CompressedImage,
6             '/camera/image/compressed', 10)
7         self.cmd_sub = self.create_subscription(Twist,

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

8         '/cmd_vel', self.cmd_vel_callback, 10)
9     self.tf_broadcaster = TransformBroadcaster(self)
10    self.timer = self.create_timer(0.02, self.timer_callback)

```

The critical ROS2 data streams flowing through this middleware layer are summarized in Table 4.

Table 4: Core ROS2 topics bridging the DJI chassis and the navigation/vision stacks.

Topic	Message Type	Rate	Direction
/odom	nav_msgs/Odometry	50 Hz	Chassis → ROS2
/camera/image/compressed	sensor_msgs/CompressedImage	10 Hz	Chassis → ROS2
/camera/camera_info	sensor_msgs/CameraInfo	10 Hz	Chassis → ROS2
/cmd_vel	geometry_msgs/Twist	On demand	ROS2 → Chassis
/scan	sensor_msgs/LaserScan	~10 Hz	LiDAR → ROS2
/tf (odom→base_footprint)	tf2_msgs/TFMessage	50 Hz	Bridge → ROS2

3.2 Chassis Navigation & DJI SDK Integration

To interface with the mobile platform, the system implements **RoboMaster-SDK-Ultra**, a high-performance community fork of the official DJI Python SDK. The robot connects to the Jetson via Wi-Fi in Station (STA) mode and is set to free-motion mode before any chassis commands are issued:

```

1  from robomaster_ultra import robot
2
3  ep_robot = robot.Robot()
4  ep_robot.initialize(conn_type="sta")
5  ep_robot.set_robot_mode(mode="free")
6  ep_chassis = ep_robot.chassis

```

Velocity commands arriving via /cmd_vel are converted to per-wheel RPM targets using the Mecanum differential mixing equations (see §6.2 for the kinematic derivation). A scaling factor of 100 maps the normalized Twist velocities to the chassis wheel-speed range:

```

1  def cmd_vel_callback(self, message):
2      vx = message.linear.x; vy = message.linear.y
3      vw = message.angular.z
4
5      left_speed  = vx - vy - vw
6      right_speed = vx + vy + vw
7      factor = 100
8
9      ep_chassis.drive_wheels(
10         w1=int(right_speed * factor), # right-front
11         w2=int(left_speed * factor),  # left-front

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

12     w3=int(left_speed * factor), # left-rear
13     w4=int(right_speed * factor)) # right-rear

```

A 300 ms communication watchdog is enforced: if no `/cmd_vel` message arrives within this window while the robot is moving, all wheel outputs are zeroed to prevent runaway behavior.

3.3 SLAM-Based Mapping Pipeline

The first phase of autonomous operation constructs a 2D occupancy grid map of the tennis court (theoretical principles in §6.4). The mapping pipeline integrates an RPLidar A1M8, the `slam_toolbox` ROS2 package, and a custom bridge node.

3.3.1 RoboMaster Chassis Bridge (map_generation.py)

The RoboMasterBridge ROS2 node publishes chassis odometry and camera data into the ROS2 ecosystem. Chassis position (x, y) and yaw θ are subscribed from the DJI SDK, converted from Euler angles to quaternions, and published at 50 Hz as TF transforms and `/odom` messages:

```

1  def timer_callback(self):
2      yaw_rad = -math.radians(c_sub.latest_yaw)
3      yaw_rad = math.atan2(math.sin(yaw_rad), math.cos(yaw_rad))
4      qx, qy, qz, qw = euler_to_quaternion(yaw_rad)
5
6      t = TransformStamped()
7      t.header.frame_id = 'odom'
8      t.child_frame_id = 'base_footprint'
9      t.transform.translation.x = -c_sub.latest_x
10     t.transform.translation.y = c_sub.latest_y
11     t.transform.rotation.x = qx; t.transform.rotation.y = qy
12     t.transform.rotation.z = qz; t.transform.rotation.w = qw
13     self.tf_broadcaster.sendTransform(t)
14
15     odom = Odometry()
16     odom.header.frame_id = 'odom'
17     odom.child_frame_id = 'base_footprint'
18     odom.pose.pose.position.x = -c_sub.latest_x
19     odom.pose.pose.position.y = c_sub.latest_y
20     self.odom_pub.publish(odom)

```

The camera stream is captured at 10 Hz, JPEG-compressed with quality factor 60, and published alongside a pinhole CameraInfo message. A 300 ms watchdog in the `/cmd_vel` subscriber ensures safety during navigation.

3.3.2 LiDAR Integration & Filtering

The RPLidar A1M8 2D laser scanner is launched via `rplidar_ros`:

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

1 ros2 launch rplidar_ros rplidar_a1_launch.py \
2   serial_port:=/dev/ttyCH341USB0

```

A `laser_filters` scan-to-scan filter chain is applied to suppress noise and constrain the effective measurement range. Static TF transforms (`base_footprint` $\rightarrow$ `base_link` $\rightarrow$ `laser`) are published to align the LiDAR frame with the robot body frame.

3.3.3 SLAM Solver (`slam_toolbox`)

The `slam_toolbox` package in online asynchronous mode fuses the filtered laser scans with chassis odometry to incrementally build a 2D occupancy grid. The operator manually teleoperates the robot around the court perimeter while the solver performs scan-to-map matching and loop closure:

```

1 ros2 launch slam_toolbox online_async_launch.py \
2   slam_params_file:=~/ros2_ws/config/mapper_params_online_async.yaml

```

Upon completion, the map is persisted to disk for subsequent reuse during autonomous navigation:

```

1 ros2 run nav2_map_server map_saver_cli -f custom_court

```

3.4 Nav2-Based Autonomous Navigation

Once the occupancy grid map is saved, the system transitions to autonomous navigation using the **Nav2** (Navigation2) framework (algorithmic details in §6.4). Nav2 provides AMCL-based localization, a global planner, a local controller, and behavior-tree-driven mission execution.

3.4.1 Localization and Planning

Nav2's Adaptive Monte Carlo Localization (AMCL) module continuously estimates the robot's pose (x, y, θ) on the pre-built map using LiDAR scan and odometry inputs. The planner server is configured with the Smac Hybrid-A* global planner and the Regulated Pure Pursuit (RPP) local controller, running at their default Nav2 parameters. The localization and navigation stacks are launched with the saved map:

```

1 # Localization
2 ros2 launch nav2_bringup localization_launch.py \
3   map:=/home/nvidia/ros2_ws/maps/custom_court.yaml \
4   use_sim_time:=False
5
6 # Navigation
7 ros2 launch nav2_bringup navigation_launch.py \
8   map:=/home/nvidia/ros2_ws/maps/custom_court.yaml \
9   use_sim_time:=False

```


Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

3.4.2 RoboMaster Navigation Bridge (map_navigation.py)

During the navigation phase, a second ROS2 node, RoboMasterNavigation, replaces the mapping bridge. Its timer_callback is nearly identical to the mapping bridge, with one critical difference: the yaw angle and position coordinates are not negated, preserving the raw chassis coordinate orientation to match the saved map's convention:

```

1 # Navigation bridge: coordinate sign differs from mapping bridge
2 yaw_rad = math.radians(c_sub.latest_yaw)      # no negation
3 corrected_x = c_sub.latest_x                  # no negation
4 corrected_y = c_sub.latest_y

```

Nav2's planner server publishes /cmd_vel directly; the bridge intercepts these messages and forwards them to the DJI chassis through the same Mecanum mixing pipeline with a 300 ms watchdog.

3.4.3 Waypoint-Based Coverage Strategy

The operator defines a sequence of navigation waypoints covering the traversable playing area in an S-shaped (boustrophedon) pattern. Nav2's behavior tree executes each waypoint sequentially. During waypoint traversal, the YOLO vision pipeline runs concurrently: if a tennis ball is detected, navigation is interrupted and the reactive retrieval pipeline (Section 4) takes over.

3.5 Video I/O & Pre-processing

The raw H.264 video feed from the DJI camera is decoded by the RoboMaster-SDK-Ultra's libmedia_codec_ultra backend. Each frame is resized to 640×360 pixels at 10 Hz and JPEG-compressed (quality 60) before being published to /camera/image/compressed. The pre-processing pipeline and full YOLO detection module are detailed in Section 4.1.

3.6 Video Processing (YOLO & OpenCV)

Real-time tennis ball detection uses the YOLO architecture with key operational parameters listed in Table 5. The system auto-detects the optimal inference backend at startup using a priority-ordered hardware probe:

```

1 DEVICE = (
2     "cuda" if torch.cuda.is_available() else
3     "mps"  if torch.mps.is_available()  else
4     "xpu"  if torch.xpu.is_available()  else
5     "cpu"
6 )

```

The detection output drives the chassis alignment controller via the horizontal pixel offset visual_error_x. The full dataset collection workflow, model training hyperparameters, and TensorRT deployment pipeline are covered in Section 4.1.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

Table 5: YOLO detection pipeline parameters.

Parameter	Value
Base model	YOLOv6-Nano (yolo26n.pt)
Inference resolution	640×360
Confidence threshold	0.25
Deployment backends	TensorRT FP16 (Jetson), PyTorch (macOS)
Nearest-target strategy	$\arg \max(y_2)$ (largest bottom y-coordinate)
Control output	$\text{visual_error_x} = \text{target_x} - \frac{\text{width}}{2}$
Frame-boundary filter	10-pixel margin from edge (relinquish lock)

3.7 Hierarchical Finite State Machine Design

The robot's behavioral logic is governed by a two-layer hierarchical Finite State Machine (formal definition in §6.5), illustrated in Figure 12. The upper layer handles iOS-originated interrupt commands with unconditional priority; the lower layer executes the autonomous mission workflow.

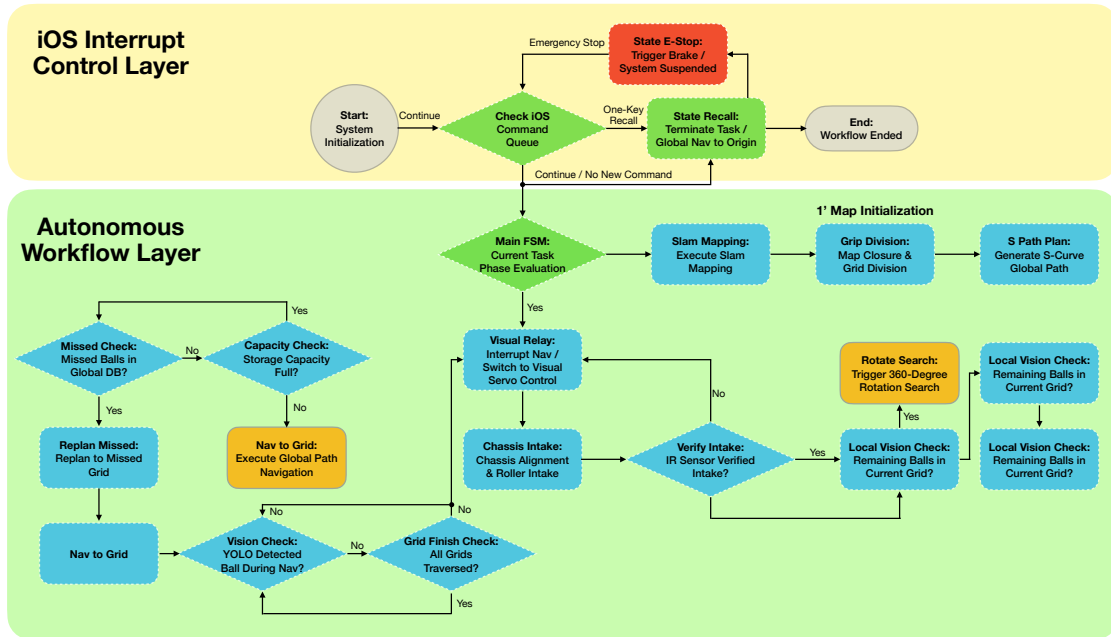


Figure 12: Hierarchical FSM flow diagram. The upper interrupt control layer preempts the lower autonomous workflow at any time. The autonomous layer progresses through three phases: mapping, waypoint navigation, and visual closed-loop retrieval with 360° post-ingestion sector sweep.

3.7.1 iOS Interrupt Control Layer (Upper FSM)

The upper FSM continuously polls the iOS command queue over UDP. Two high-priority interrupt commands preempt any ongoing autonomous behavior:

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

1. **Emergency Stop (E-STOP):** Immediately engages chassis braking and suspends all autonomous logic until the operator issues a resume command.
2. **One-Touch Recall:** Terminates the current mission and invokes the Nav2 planner to navigate the robot to the predefined origin pose.

3.7.2 Autonomous Workflow Layer (Lower FSM)

The lower FSM transitions through three sequential mission phases:

Phase 1 — Mapping & Global Planning. SLAM toolbox constructs the occupancy grid while the operator teleoperates the robot around the court perimeter. Once the map is saved, an S-shaped waypoint sequence is loaded into the Nav2 waypoint follower.

Phase 2 — Waypoint Navigation & Interrupt Handling. Before dispatching each waypoint, two pre-navigation checks execute: a missed-ball database lookup for previously detected but unrecovered ball locations, and a storage capacity check that triggers recall if the onboard bay is full. If clear, Nav2 navigates to the next waypoint. During traversal, the YOLO pipeline runs concurrently; upon detection, navigation is interrupted and Phase 3 activates. When all waypoints are visited, the system triggers a recall.

Phase 3 — Visual Closed-Loop Retrieval. This phase is implemented in `chassis_ctrl.py` as an implicit state machine driven by three Boolean flags defined at module level:

```

1 visual_error_x = 0
2 target_locked  = False      # whether a ball is currently locked
3 scan_needed    = False      # whether to perform a 360-degree scan
4 no_ball_counter = 0         # consecutive frames without detection

```

The main control loop transitions between behavioral modes based on these flags and the real-time IR distance sensor reading (millimetres). The transition logic at the top of the loop is:

```

1 target_valid = vn.target_locked
2 need_scan    = vn.scan_needed
3 dist = ds.latest_distance if ds.latest_distance is not None else 8848
4
5 if target_valid:
6     is_chasing_sequence = True
7     last_valid_dist = dist
8 elif need_scan:
9     is_chasing_sequence = False
10    last_valid_dist = 8848

```

When `is_chasing_sequence` is active, the controller executes three priority-ordered behaviors based on the current sensor reading. Key numerical thresholds are listed in Table 6.

After confirmed ingestion, the robot executes a 360° in-place rotational scan. If YOLO detects another ball during the scan, the chasing sequence re-engages (looping back to visual servoing). If the scan completes with no detections, the sector is marked as cleared, the global ball-coordinate database is updated, and control returns to the main FSM for the next waypoint dispatch. The detailed implementation of the reactive control pipeline is presented in Section 4.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

Table 6: FSM Phase 3 operational parameters implemented in `chassis_ctrl.py` and `video_node.py`.

Stage	Parameter	Value
Target acquisition	Consecutive no-ball frames to trigger scan	15
Visual servoing	Turn PID (<code>pid_turn</code>)	$K_p = 0.15, K_i = 0.03, K_d = 0.03$
	Turn PID output limits	$[-50, 50]$
	Forward PID (<code>pid_forward</code>)	$K_p = 0.5, K_i = 0.1, K_d = 0.05$
	Forward PID output limits	$[-40, 40]$
	Alignment-priority threshold	$ \text{error}_x > 40 \text{ px}$
Blind-zone relay	Open-loop cruise speed	30 (forward)
	Target-lost distance guard	$\text{dist} < 450 \text{ mm}$
	Last-valid proximity guard	$\text{last_valid_dist} < 300 \text{ mm}$
Ingestion trigger	Pre-spin distance threshold	$\leq 200 \text{ mm}$
	Motor pre-spin duty	90 ($\sim 35\%$)
	Approach speed during ingestion	30 (forward)
Ingestion confirm	Disappearance distance threshold	$> 250 \text{ mm}$
	Consecutive disappearance frames	10 (200 ms)
	Hard timeout	3.0 s
360° sector sweep	In-place rotation speed	30 (turn)

3.8 Class Design

CourtNavigationManager Class Description

The central orchestrator of the autonomous workflow is the `CourtNavigationManager` class, which bridges the ROS2 navigation stack, the computer vision pipeline, and the hardware actuation interfaces through the hierarchical FSM. At the implementation level, the Phase 3 reactive FSM logic resides in `chassis_ctrl.py` using the Boolean flag mechanism described above; `CourtNavigationManager` coordinates the higher-level phase transitions and Nav2 dispatch.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

Attribute	Description
Class Name	CourtNavigationManager
Aggregated Classes	YoloDetector, Nav2PlannerClient, DjiChassisController, MotorPwmDriver, IrSensorReader
Associated Classes	MjpegStreamServer, IosCommandReceiver (UDP listener for iOS interrupts)
Data Members	• <code>master_state</code>: Enum (MAPPING, NAVIGATING, INTERCEPTING, RETRIEVING, RECALLING, E_STOP). • <code>robot_pose_global</code>: Pose (x, y, θ) obtained from Nav2 AMCL localization. • <code>waypoint_queue</code>: Ordered S-shaped sequence of navigation goals. • <code>missed_ball_db</code>: List of (x, y) coordinates of detected but unrecovered balls. • <code>storage_capacity</code>: Current ball count in the onboard bay.
Member Functions	• <code>process_ios_interrupt()</code>: Receives UDP commands from the iOS app (E-STOP / Recall / Continue). • <code>update_master_state()</code>: High-level FSM transitions based on vision events, waypoint progress, and capacity thresholds. • <code>generate_coverage_path()</code>: Generates S-shaped waypoint sequence from the SLAM map. • <code>dispatch_nav2_goal()</code>: Sends a target pose to Nav2 and monitors execution. • <code>handle_visual_intercept()</code>: Suspends Nav2, switches to PD visual servoing, triggers intake on proximity confirmation. • <code>perform_360_scan()</code>: In-place rotation with YOLO monitoring; loops back to retrieval if a new ball is found.
Processing	Operates in a dedicated high-priority thread. Fuses Nav2 pose with YOLO detections, executes the hierarchical FSM, dispatches velocity commands, and manages motor triggering.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

4 SOFTWARE DESIGN: PERCEPTION AND REACTIVE CONTROL

4.1 Computer Vision & Object Detection Module

This module is the core perception layer of the CourtSweeper system, responsible for real-time video frame capture, tennis ball target detection, and providing pixel-space coordinate offset to the downstream motion control module. The system is built on the YOLO (You Only Look Once) deep learning object detection framework, supports multi-platform deployment (macOS, Linux/Jetson), and is optimized with TensorRT acceleration for embedded edge computing scenarios. Theoretical foundations — including the camera pinhole model, IBVS error, YOLO architecture and loss functions, and LiDAR triangulation — are presented in §6.1.

4.1.1 Module Architecture

The overall workflow of the computer vision module can be summarized into four stages: dataset collection and annotation, model training, cross-platform model deployment, and real-time object detection. The system receives the real-time video stream from the RoboMaster gimbal camera, performs YOLO inference to output the bounding box coordinates of tennis ball targets, and then computes the horizontal offset of the target relative to the image center (visual error), which serves as the input signal for the chassis and gimbal control loops.

The module consists of the following core files:

<code>cv_config.py</code>	Device-adaptive and path configuration
<code>dataset_collect.py</code>	Dataset collection tool
<code>model_training.py</code>	YOLO model training script
<code>pt2onnx.py</code>	PyTorch model export to ONNX format
<code>onnx2engine.sh</code>	ONNX model conversion to TensorRT engine
<code>video_node.py</code>	Real-time object detection main node

4.1.2 Dataset Collection

The quality of the dataset directly affects the detection accuracy of the model. We developed the `dataset_collect.py` tool to capture tennis ball images in actual court environments using the built-in RoboMaster camera.

The collection program communicates with the robot via a direct Wi-Fi connection (AP mode), starts a 720P HD video stream, and displays the resized frames at 1280×960 pixels in real time. A crosshair is drawn at the center of the frame to assist composition, helping the collector position the tennis ball in the central region. Pressing the `s` key triggers a snapshot save, followed by a brief white flash as visual feedback; pressing `q` exits the program. Images are saved to the `./dataset/tennis_v2/` directory with a timestamp-plus-sequence naming scheme (e.g., `tennis_20250519_143025_0001.jpg`).

```
1 def img_collect():
```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

2  save_dir = "./dataset/tennis_v2"
3  os.makedirs(save_dir, exist_ok=True)
4
5  ep_robot = robot.Robot()
6  ep_robot.initialize(conn_type="ap")
7  ep_robot.set_robot_mode(mode="free")
8  ep_camera = ep_robot.camera
9  ep_camera.start_video_stream(display=False,
10                               resolution=camera.STREAM_720P)
11
12  count = 0
13  while True:
14      img = ep_camera.read_cv2_image()
15      if img is not None:
16          img_resized = cv2.resize(img, (1280, 960))
17          cv2.line(display_img, (center_w - 20, center_h),
18                  (center_w + 20, center_h), (0, 255, 0), 2)
19          cv2.line(display_img, (center_w, center_h - 20),
20                  (center_w, center_h + 20), (0, 255, 0), 2)
21          if key == ord('s'):
22              cv2.imwrite(filename, img_resized)
23

```

After collection, annotation tools such as LabelImg are used to label the tennis balls in the images with bounding boxes, generating YOLO-format label files (.txt). A data.yaml dataset configuration file is then written, specifying the training and validation set paths and class names.

4.1.3 Model Training

The object detection model is trained using the Ultralytics YOLO framework. Starting from the YOLOv6-Nano (yolo26n.pt) pre-trained weights as a foundation, transfer learning is applied to rapidly adapt the model from general-purpose detection capability to the single tennis ball class.

The training hyperparameters are configured as follows:

Initial learning rate (lr0)	0.01
final learning rate (lrf)	0.01
Optimizer	auto (SGD with momentum=0.937)
Weight decay	0.0005
Warmup epochs	3.0
Data augmentation	Mosaic stitching, random horizontal flip (fliplr=0.5), HSV color perturbation, random affine transformation (scale=0.5, translate=0.1), RandAugment automatic augmentation, random erasing (erasing=0.4)
Early stopping patience	100 epochs

```

1  from ultralytics import YOLO

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

2
3 def train_model():
4     model = YOLO("./weight/yolo26n.pt")    # load pre-trained model
5
6     results = model.train(
7         data=DATA_YAML,                    # dataset configuration file
8         epochs=200,                        # training epochs
9         imgsz=640,                         # input image size
10        batch=16,                          # batch size (adjust by GPU memory)
11        name=MODEL_NAME,                   # model name
12        device=DEVICE,                     # runtime device
13        workers=2,                         # data loading threads
14        project="./runs/train_tennis",     # output directory
15        exist_ok=False                     # do not overwrite existing experiment
16    )

```

Device selection is handled by the adaptive logic in `cv_config.py`, which probes hardware accelerators in priority order: CUDA (NVIDIA GPU), MPS (Apple Silicon), XPU (Intel GPU), and falls back to CPU if none is available:

```

1 DEVICE: str = (
2     "cuda" if torch.cuda.is_available() else
3     "mps" if torch.mps.is_available() else
4     "xpu" if torch.xpu.is_available() else
5     "cpu"
6 )

```

The artifacts produced during training include: the best-weight file `best.pt`, the last-epoch weight `last.pt`, F1 curve plot, PR curve plot, confusion matrix, training batch visualization samples, and a `results.csv` file recording per-epoch losses and metrics for subsequent analysis and tuning.

4.1.4 Cross-Platform Model Deployment

To accommodate the inference performance requirements of different deployment environments, the system implements a model conversion pipeline from PyTorch to ONNX to TensorRT.

PyTorch → ONNX Conversion: Using the built-in export functionality of Ultralytics, the `.pt` weights can be converted to the open ONNX intermediate representation format with a single line of code:

```

1 from ultralytics import YOLO
2
3 model = YOLO("./pt/tennis.pt")
4 model.export(format="onnx")

```

The generated `.onnx` file can be used for inference on any platform supporting ONNX Runtime, ensuring the model's universality and portability.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

ONNX → TensorRT Engine: On edge computing platforms such as the NVIDIA Jetson, to fully leverage GPU inference acceleration, NVIDIA TensorRT's `trtexec` tool is used to compile the ONNX model into a highly optimized TensorRT engine (`.engine` file), with FP16 half-precision inference enabled to substantially boost the inference frame rate with negligible loss in detection accuracy:

```

1 /usr/src/tensorrt/bin/trtexec      \
2 --onnx=onnx/tennis_v2.onnx        \
3 --saveEngine=engine/tennis_v2.engine \
4 --fp16

```

At inference time, the appropriate model format is automatically selected based on the run-time platform:

```

1 # Mac Config
2 model = YOLO("src/vision/mlmodel/pt/tennis_v2.pt")
3
4 # Linux Config (Jetson + TensorRT)
5 # model = YOLO("src/vision/mlmodel/engine/tennis_v2.engine")
6 # model.names = {0: 'tennis ball'}

```

4.1.5 Real-Time Object Detection Pipeline

`video_node.py` is the runtime core of the entire vision module. It is responsible for acquiring video frames from the robot camera, invoking the YOLO model for object detection, computing the target's offset from the frame center, and overlaying rich debugging information on the frame.

Video Stream Acquisition.

The video stream is started at 360P resolution via the RoboMaster SDK to balance image quality and frame rate. Each received frame is resized to 640×360 pixels before being fed into the YOLO model for processing.

```

1 def video_capture(ep_camera):
2     global latest_frame, annotated_frame, running
3
4     ep_camera.start_video_stream(display=False,
5                                   resolution=camera.STREAM_360P)
6
7     while running:
8         img = ep_camera.read_cv2_image()
9         if img is not None:
10             img = cv2.resize(img, (640, 360))
11             latest_frame = img
12             annotated_frame = yolo_predict(img)
13             cv2.imshow("on Live", annotated_frame)

```


Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

Object Detection and Filtering.

YOLO inference uses a confidence threshold of 0.25 to filter out low-quality detections. When multiple detection results are present in the frame (e.g., multiple tennis balls), the system compares the bottom Y-coordinate (y2 value) of each bounding box to select the ball closest to the camera — a larger y2 value indicates that the target is lower in the frame, i.e., nearer to the camera. This strategy ensures that the robot prioritizes locking onto and tracking the nearest target ball.

Once the target is selected, the horizontal coordinate of its bounding box center, `target_x`, is subtracted from the frame centerline `center_x` to compute the horizontal visual deviation `visual_error_x`. This offset is the key input quantity for subsequent chassis rotation control:

$$\text{visual_error_x} = \text{target_x} - \frac{\text{width}}{2}$$

```

1  def yolo_predict(frame):
2      results = model(frame, conf=0.25, device=cfg.DEVICE,
3                      verbose=False)
4      annotated = results[0].plot()
5
6      height, width = frame.shape[:2]
7      center_x = width // 2
8
9      if len(results[0].boxes) > 0:
10         boxes = results[0].boxes
11
12         closest_box = None
13         max_y2 = -1
14         for box in boxes:
15             x1, y1, x2, y2 = map(int, box.xyxy[0])
16             if y2 > max_y2:
17                 max_y2 = y2
18                 closest_box = box # select the nearest target
19
20         x1, y1, x2, y2 = map(int, closest_box.xyxy[0])
21         target_x = (x1 + x2) // 2
22         target_y = (y1 + y2) // 2
23         visual_error_x = target_x - center_x

```

Target Locking and Scanning Mechanism.

The system maintains three states: Locked (target locked), Searching (normal search in progress), and Scanning (360-degree scan for ball search). When no tennis ball is detected for 15 consecutive frames (`no_ball_counter >= 15`), the system automatically triggers the `scan_needed` flag, notifying the chassis control module to perform an in-place rotation scan to re-locate the ball. Upon detecting a target, the counter resets to zero and the state returns to locked. Additionally, if the target approaches the edge of the frame (within 10 pixels of the boundary), the system treats it as about to leave the field of view and actively relinquishes the lock to avoid ineffective tracking.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

1  if not (10 <= target_x <= width - 10) or \
2      not (10 <= target_y <= height - 10):
3      target_x = None
4      target_y = None
5      visual_error_x = 0
6      target_locked = False
7
8  # no ball detected
9  no_ball_counter += 1
10 if no_ball_counter >= 15: # 15 consecutive frames without target → trigger scan
11     ↪ mode
12     scan_needed = True

```

4.1.6 Visual Overlay & Debug Information

To facilitate real-time debugging and status monitoring, the system overlays the following information on each frame in OSD (On-Screen Display) format:

- **Resolution (RES):** Current frame dimensions, displayed in red
- **Frame Rate (FPS):** Sliding-average frame rate over the last 10 frames, displayed in green
- **Distance:** Real-time ranging value from the distance sensor module (unit: cm), displayed in blue
- **Status:** Current tracking state — Locked (green) / Searching or Scanning (orange)
- **Center Crosshair:** A gray crosshair marking the geometric center of the frame
- **Target Marker:** When a target is locked, a solid blue circle is drawn at the target center with the horizontal offset value annotated beside it
- **Bounding Box:** YOLO detection boxes are rendered as yellow rectangles

```

1  # Resolution
2  cv2.putText(annotated, f"RES: {width}x{height}",
3              (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
4              1, (0, 0, 255), 2, lineType=cv2.LINE_AA)
5  # Frame Rate
6  cv2.putText(annotated, f"FPS: {int(avg_fps)}",
7              (10, 60), cv2.FONT_HERSHEY_SIMPLEX,
8              1, (0, 255, 0), 2, lineType=cv2.LINE_AA)
9  # Distance
10 cv2.putText(annotated, f"Distance: {distance_text}",
11             (10, 90), cv2.FONT_HERSHEY_SIMPLEX,
12             1, (255, 0, 0), 2, lineType=cv2.LINE_AA)
13 # Status
14 state_text = "Locked" if target_locked else \
15             ("Scanning" if scan_needed else "Searching")

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

16 cv2.putText(annotated, f"Status: {state_text}",
17             (10, 120), cv2.FONT_HERSHEY_SIMPLEX,
18             1, color, 2, lineType=cv2.LINE_AA)

```

4.1.7 Inter-Module Data Exchange

The computer vision module shares data with other system modules through module-level global variables:

- `visual_error_x`: The horizontal pixel offset of the target relative to the frame center, used by the chassis control module (`chassis_ctrl.py`) to compute the rotational angular velocity
- `target_locked`: The target lock status flag, used by the main control logic to determine the current tracking phase
- `scan_needed`: The 360-degree scan ball-search request flag, triggering the chassis to execute a scanning rotation maneuver
- `latest_frame` / `annotated_frame`: The raw and annotated frames, available for other modules that require image data

Simultaneously, the vision module also reads the `latest_distance` variable provided by the distance sensor module, displaying real-time tennis ball distance information on the OSD and thereby achieving fused multi-sensor information presentation.

4.1.8 Summary

The CourtSweeper computer vision module utilizes the YOLO deep learning framework to achieve a complete closed-loop pipeline encompassing data collection, model training, cross-platform deployment, and real-time inference. Through the nearest-target filtering strategy, target locking and scanning search mechanism, and horizontal visual deviation computation, the module provides reliable perceptual input for the robot's autonomous tracking and ball retrieval behaviors. The model deployment pipeline supports the three-stage PyTorch→ONNX→TensorRT conversion, enabling efficient inference at FP16 half-precision on edge devices such as the NVIDIA Jetson, thereby satisfying real-time performance requirements.

4.2 Intake Motor Control Module

The intake motor control module is the critical actuation unit of the CourtSweeper system. It is responsible for driving the dual-roller mechanical structure to ingest the ball into the onboard storage bay once the vision system has locked onto a tennis ball target. This module spans both low-level hardware driving and high-level decision logic, forming a complete control chain from signal generation to intelligent triggering. The underlying PWM theory, BTS7960 H-bridge topology, and watchdog mechanism are formalised in §6.8.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

4.2.1 Hardware Driver Architecture

The intake mechanism is driven by two 775 DC motors (D-shaft configuration, rated at 12 V and 4600 RPM), each coupled to a 60 mm high-friction solid rubber roller via a 30 mm extended brass coupling. Motor driving is handled by two BTS7960 high-current full-bridge driver modules (capable of handling up to 43 A peak current), independently controlling the direction and speed of the left and right rollers.

The BTS7960 driver modules interface with the lower-level controller via a three-wire interface (enable pin EN, forward PWM pin RPWM, and reverse PWM pin LPWM). The system utilizes a dedicated 3S (11.1 V) 5200 mAh Lithium Polymer (LiPo) battery (discharge rate 40C) to power the driver modules, ensuring sufficient burst current for the motors during ball compression while preventing voltage sag interference to the main control logic circuits. Low-level PWM signal generation is handled by an Arduino microcontroller, which receives commands from the Jetson Orin NX via USB serial.

4.2.2 Low-Level Firmware Design

The low-level Arduino firmware (`roller_setting.ino`) is responsible for converting serial commands into PWM signals, enabling real-time control of the dual BTS7960 drivers. The firmware defines the control interfaces for the left and right motors through a pin-mapping scheme, as detailed in Table 7.

Table 7: 775 Motor Driver Pin Mapping Table

Motor	Enable (EN)	Forward PWM (RPWM)	Reverse PWM (LPWM)
Left Motor	Pin 4	Pin 5	Pin 6
Right Motor	Pin 7	Pin 9	Pin 10

The firmware main loop operates in blocking serial-read mode: when data is available in the serial buffer, it calls `Serial.parseInt()` to continuously parse the left and right motor speed values (formatted as "`L_speed,R_speed\n`"), then invokes the `driveMotor()` function to convert the values into corresponding PWM signals. This function clamps the speed to the range $[-255, 255]$ and determines the rotation direction based on the sign: positive values activate the RPWM channel, negative values activate the LPWM channel, and a zero value pulls the enable pin low to cut off the output.

```

1 void driveMotor(int enPin, int fwdPin, int revPin, int speed) {
2   speed = constrain(speed, -255, 255);
3
4   if (speed == 0) {
5     digitalWrite(enPin, LOW);
6     analogWrite(fwdPin, 0);
7     analogWrite(revPin, 0);
8   } else if (speed > 0) {
9     digitalWrite(enPin, HIGH);
10    analogWrite(fwdPin, speed);

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

11     analogWrite(revPin, 0);
12 } else {
13     digitalWrite(enPin, HIGH);
14     analogWrite(fwdPin, 0);
15     analogWrite(revPin, abs(speed));
16 }
17 }

```

To ensure system safety, the firmware incorporates a communication timeout protection mechanism. Upon each successful command parsing, a timestamp is recorded. If no new command is received within 500 ms while the motors are in motion, `stopMotor()` is automatically invoked to zero all motor outputs, preventing the motors from running idle during a communication loss.

```

1 void loop() {
2     if (Serial.available() > 0) {
3         int speedL = Serial.parseInt();
4         int speedR = Serial.parseInt();
5
6         if (Serial.read() == '\n') {
7             driveMotor(L_EN, L_RPWM, L_LPWM, -speedL);
8             driveMotor(R_EN, R_RPWM, R_LPWM, speedR);
9
10            lastRecvTime = millis();
11
12            if (speedL != 0 || speedR != 0) {
13                isMoving = true;
14            }
15        }
16    }
17
18    if (isMoving && (millis() - lastRecvTime > 500)) {
19        stopMotor();
20        isMoving = false;
21    }
22 }

```

4.2.3 High-Level Control Interface

On the Jetson Orin NX side, the Python module `roller_drive.py` encapsulates serial communication with the Arduino. Its core is the `RollerMotor` class: the constructor establishes a connection via the `PySerial` library at 115200 baud and waits 2 s for initialization, while the `set_speed()` method formats the left and right wheel speeds as a CSV string and writes it to the serial port.

```

1 class RollerMotor:
2     def __init__(self, port='', baudrate=115200):
3         logger.info(f"Connecting to motor on {port} ...")

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

4      try:
5          self.ser = serial.Serial(port, baudrate, timeout=1)
6          time.sleep(2)
7          logger.info("Connection successful!")
8      except Exception as e:
9          logger.error(f"Failed to connect to serial port. Error details: {e}")
10         exit(1)
11
12     def set_speed(self, left_speed, right_speed):
13         command = f"{int(left_speed)},{int(right_speed)}\n"
14         self.ser.write(command.encode('utf-8'))
15         logger.info(f"Roller Speed: L = {left_speed}, R = {right_speed}")
16
17     def close(self):
18         self.set_speed(0, 0)
19         time.sleep(0.1)
20         self.ser.close()
21         logger.info("Connection closed.")

```

This design treats the Python side as a pure command-dispatching layer, offloading all PWM timing generation and direction logic to the Arduino firmware, allowing the upper layer to focus on decision scheduling without concern for time-sensitive low-level signal generation. The `close()` method ensures safe shutdown by issuing a zero-speed command before closing the serial port.

4.2.4 FSM Integration & Intake Trigger Logic

The motor control module is integrated into the chassis control main loop (`chassis_ctrl.py`) and is triggered by the Finite State Machine (FSM) based on visual perception results. The triggering and execution of the intake logic can be decomposed into the following three phases:

Phase 1: Target Distance Pre-Assessment

When the vision system locks onto a tennis ball target (`target_locked = True`), the system enters the chasing sequence (`is_chasing_sequence`). The PD controller drives the chassis to continuously align with and approach the target, while simultaneously reading the front-facing distance sensor value in real time. When the target distance falls within 200 mm, the system determines that the ball has entered the reachable range of the rollers and immediately transitions to the pre-spin phase.

Phase 2: Motor Pre-Spin & Slow Advance

Upon entering the pre-spin phase, the motors begin rotating at a medium speed (both set to 90, approximately 35% duty cycle), while the chassis continues a slow forward advance at speed 30. This design ensures that the rollers already possess rotational kinetic energy at the moment of contact with the ball, preventing jamming or bouncing caused by acceleration from a standstill.

```

1  if dist <= 200:
2      logger.info(f"      ({dist/10:.1f}cm)      ...")

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

3
4     if intake_motor:
5         intake_motor.set_speed(90, 90)
6
7     drive_chassis(ep_chassis, forward_speed=30, turn_speed=0)

```

Phase 3: Ingestion Confirmation & Timeout Protection

After the motors start, the system enters an ingestion confirmation window of at most 3 s. The front-facing distance sensor value is continuously monitored at a 20 ms sampling period: if the ball is successfully ingested, the distance sensor reading will jump beyond the measurement range (> 250 mm). The system performs 10 consecutive disappearance confirmations (totaling 200 ms) to prevent false positives. Once the ball is confirmed ingested, it immediately stops the chassis and motors and resets the chasing state, returning the FSM to the searching state to resume the coverage path.

If the distance sensor reading does not exhibit any jump within the 3 s window, the system determines that a jam or missed-ball anomaly has occurred and likewise stops execution while outputting a warning log, preventing motor overheating from prolonged stalling.

```

1  swallow_start_time = time.time()
2  missing_count = 0
3
4  while time.time() - swallow_start_time < 3.0:
5      current_dist = ds.latest_distance if ds.latest_distance is not None else 8848
6
7      if current_dist > 250:
8          missing_count += 1
9      else:
10         missing_count = 0
11
12     if missing_count >= 10:
13         is_swallowed = True
14         break
15
16     time.sleep(0.02)
17
18 if is_swallowed:
19     logger.info(" ")
20 else:
21     logger.info(" ")
22
23 chassis_stop(ep_chassis)
24
25 if intake_motor:
26     intake_motor.set_speed(0, 0)

```

Blind-Zone Relay Mechanism

Additionally, the system implements a blind-zone relay logic: when the ball distance has already been reduced to within 300 mm but the visual target is briefly lost (with the distance sensor still

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

reading within the valid 450 mm range), the chassis maintains straight forward advance without immediately aborting the chasing sequence. This mechanism effectively compensates for the visual blind zone caused by the camera’s limited field of view at close range, improving the success rate of near-field ingestion.

Post-Ingestion Sector Sweep and FSM Re-Entry

Following a confirmed ball ingestion, the system does not immediately resume waypoint navigation. Instead, it executes a 360° in-place rotational scan at a fixed angular velocity (30 RPM) to check whether additional tennis balls remain within the current court sector. During the rotation, the YOLO pipeline runs continuously: if a new ball is detected, the chasing sequence re-engages, pulling execution back to the visual servoing phase. This looping mechanism ensures that all balls within a local sector are retrieved before the robot moves on, avoiding expensive back-and-forth navigation.

If the rotation completes with no further detections, the system determines that the current sector has been fully cleared. The sector status is updated in the global ball-coordinate database, and control returns to the main FSM. The FSM then dispatches the next waypoint from the Nav2 coverage path (or triggers a recall if all waypoints have been visited), as described in the hierarchical FSM design in Section 3.7.

4.3 Proximity-Based Chassis Navigation Module

This module combines real-time proximity sensing with a closed-loop PID chassis controller, responsible for executing the reactive retrieval stage of ball capture: alignment, approach, and physical ingestion. It bridges the visual perception pipeline (Section 4.1) with the intake motor actuator (Section 4.2) through the phase-3 reactive control logic of the hierarchical FSM (Section 3.7), simultaneously fusing two independent data streams — the image-plane centroid offset from the YOLO detector, and millimeter-resolution distance readings from the RoboMaster EP’s onboard infrared sensor. The Mecanum kinematics, dual-PID control laws, and blind-zone relay logic are derived in the Theoretical Manual (§6.2–§6.3).

4.3.1 Distance Sensing Submodule

Hardware Interface via RoboMaster-SDK-Ultra.

Distance measurement relies on the RoboMaster EP’s built-in infrared proximity sensor, accessed through **RoboMaster-SDK-Ultra**. This SDK is a community-maintained high-performance fork of the official DJI Python SDK, extending compatibility to Python 3.7 through 3.13 and bundling `libmedia_codec_ultra` for zero-copy H.264 decoding. Installation on the Jetson Orin NX proceeds as follows:

```

1 # Automated installation (recommended)
2 git clone https://github.com/RamessesN/Robomaster-SDK-Ultra
3 cd Robomaster-SDK-Ultra
4 sudo chmod 755 ./installer.sh && ./installer.sh
5
6 # Or manual installation on Ubuntu/Debian

```


Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

7  sudo apt update && sudo apt install ffmpeg libopus-dev
8  pip install -e .
9  cd robomaster_lib/libmedia_codec_ultra && pip install -e .
10 cd ../../pybind11 && pip install -e .

```

After installation, the sensor API remains consistent with the official DJI documentation, but the import must come from the patched namespace:

```

1  from robomaster_ultra import robot    # replaces: from robomaster import robot

```

Subscription Design.

Distance data is obtained through the SDK's asynchronous subscription callback mechanism. A dedicated background thread invokes `sub_distance(freq=10, callback=...)` to register a 10 Hz polling loop; upon each callback trigger, the raw sensor reading (unit: millimetres) is written to a module-level shared variable:

```

1  # src/distance/distance_sub.py
2
3  latest_distance: float | None = None
4
5  def get_distance(ep_sensor):
6      """Register the 10 Hz distance subscription."""
7      ep_sensor.sub_distance(freq=10, callback=sub_data_handler_distance)
8
9  def sub_data_handler_distance(sub_info):
10     """Callback: atomically update the shared distance register."""
11     global latest_distance
12     latest_distance = sub_info[0]    # unit: millimetres

```

The sentinel value `None` is used to distinguish “sensor not yet initialized” from a legitimate zero reading. In all chassis controller logic, a `None` reading is substituted with the symbolic constant 8848 (the height of Mount Everest in metres, employed here as an equivalent infinitely-far distance value), ensuring the controller defaults to a pure vision-guided mode in the absence of sensor data.

4.3.2 PID Controller Configuration

Two independent PID controllers respectively govern the two degrees of freedom required during the approach process:

```

1  from simple_pid import PID
2
3  # Lateral alignment: drives image-plane centroid error to zero
4  pid_turn = PID(0.15, 0.03, 0.03, setpoint=0)
5  pid_turn.output_limits = (-50, 50)    # RPM-equivalent units

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

6
7 # Longitudinal approach: drives sensor distance to 10 mm (contact)
8 pid_forward = PID(0.5, 0.1, 0.05, setpoint=10)
9 pid_forward.output_limits = (-40, 40)

```

- **pid_turn** receives the pixel-space lateral offset `visual_error_x` produced by the YOLO centroid extractor (a positive value indicates the target is to the right of the frame center). Its output is negated before application, such that a target on the right produces a clockwise chassis rotation.
- **pid_forward** receives the infrared distance reading (millimetres) and drives the robot toward a target distance of 10 mm — i.e., physical contact with the ball. When the distance is below 800 mm, this controller overrides the open-loop cruise speed (30 RPM); above 800 mm, the chassis advances at a fixed cruise speed to reduce approach time.

4.3.3 Chasing Sequence State Tracking

The Boolean flag `is_chasing_sequence` decouples the visual locking phase from the physical approach phase. Once the YOLO pipeline sets `target_locked` to `True`, this flag is latched high, and the last valid distance reading is saved to `last_valid_dist`. The flag is cleared only when the vision module issues `scan_needed = True` — indicating complete target loss following a successful ingestion or timeout.

This design prevents the distance sensor from falsely triggering the intake motors due to irrelevant obstacles briefly entering the sensor field of view before a visual lock has been established.

```

1 if target_valid:
2     is_chasing_sequence = True
3     last_valid_dist = dist
4 elif need_scan:
5     is_chasing_sequence = False
6     last_valid_dist = 8848

```

4.3.4 Multi-Stage Approach & Ingestion Logic

When `is_chasing_sequence` is active, the controller executes three priority-ordered behaviors based on the current sensor reading.

Phase 1 — Cruise Tracking (`dist > 800 mm`).

Both PID loops operate simultaneously. The turn controller corrects lateral deviation; when the centroid error exceeds 40 pixels, the forward speed is limited to 5 RPM to ensure rotational alignment stabilizes before resuming advance:

```

1 turn_speed = -pid_turn(error_x)
2 if abs(error_x) > 40:
3     forward_speed = 5           # prioritise alignment over advance

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

4 else:
5     if dist < 800:
6         forward_speed = -pid_forward(dist)    # PID-controlled approach
7     else:
8         forward_speed = 30    # open-loop cruise
9 drive_chassis(ep_chassis, forward_speed, turn_speed)

```

Phase 2 — Blind-Zone Relay (target lost and dist < 450 mm).

The camera's minimum focusing distance together with the physical offset between the camera and the intake rollers collectively form a "blind zone" — the ball disappears from the video frame before entering the rollers. The relay logic bridges this blind zone: if the target was last confirmed within 300 mm (`last_valid_dist < 300`) and the distance sensor still reads below 450 mm, the chassis maintains straight forward advance at 30 RPM without visual feedback:

```

1 elif not target_valid and dist < 450 and last_valid_dist < 300:
2     logger.info(
3         f"Blind-zone relay "
4         f"(current: {dist/10:.1f} cm, "
5         f"last valid: {last_valid_dist/10:.1f} cm) "
6         "-- maintaining straight advance..."
7     )
8     drive_chassis(ep_chassis, forward_speed=30, turn_speed=0)

```

Phase 3 — Ingestion Trigger (dist ≤ 200 mm).

When the sensor confirms the ball has entered the 200 mm range, the intake motors pre-spin at full speed and the chassis advances at a fixed 30 RPM. A sliding-window detector then continuously monitors the ball disappearance signal: if the distance reading exceeds 250 mm for 10 consecutive 20 ms cycles (`missing_count ≥ 10`), the ball is determined to have been successfully ingested. A 3-second hard timeout guards against jamming or missing:

```

1 if dist <= 200:
2     logger.info(
3         f"Target entered pre-ingestion zone "
4         f"({dist/10:.1f} cm) -- pre-spinning motors..."
5     )
6     if intake_motor:
7         intake_motor.set_speed(90, 90)
8     drive_chassis(ep_chassis, forward_speed=30, turn_speed=0)
9
10    swallow_start_time = time.time()
11    missing_count = 0
12    is_swallowed = False
13
14    while time.time() - swallow_start_time < 3.0:
15        current_dist = ds.latest_distance \
16            if ds.latest_distance is not None else 8848

```

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

```

17         if current_dist > 250:
18             missing_count += 1
19         else:
20             missing_count = 0
21         if missing_count >= 10:
22             is_swallowed = True
23             break
24         time.sleep(0.02)
25
26     if is_swallowed:
27         logger.info("Ball successfully ingested into storage bay.")
28     else:
29         logger.info("Warning: advance timeout -- possible jam or miss.")
30
31     chassis_stop(ep_chassis)
32     if intake_motor:
33         intake_motor.set_speed(0, 0)
34     is_chasing_sequence = False
35     last_valid_dist = 8848
36     time.sleep(0.5)
37     continue

```

4.3.5 Chassis Drive Primitives

The Mecanum wheel drive equation converts the two scalar commands `forward_speed` and `turn_speed` into per-wheel RPM targets. Since the DJI SDK's `drive_wheels(w1, w2, w3, w4)` interface uses the right-front / left-front / left-rear / right-rear wheel ordering convention, the differential mixing is computed as follows:

```

1  def drive_chassis(ep_chassis, forward_speed, turn_speed):
2      left_speed = forward_speed + turn_speed
3      right_speed = forward_speed - turn_speed
4      ep_chassis.drive_wheels(
5          w1=right_speed,    # right-front
6          w2=left_speed,     # left-front
7          w3=left_speed,     # left-rear
8          w4=right_speed     # right-rear
9      )
10
11  def chassis_stop(ep_chassis):
12      ep_chassis.drive_wheels(w1=0, w2=0, w3=0, w4=0)
13      time.sleep(0.02)      # brief settle pause before next command

```

During pure straight-line motion, the two sides receive equal speeds; when `turn_speed` is non-zero, one side's speed increases while the other decreases, producing an in-place rotation superimposed on the translational velocity. The 20 ms settling delay in `chassis_stop` prevents new burst commands from being issued before the previous wheel-speed packet is confirmed by the chassis firmware.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

4.3.6 Idle & Search Behavior

When no tracking task is in progress, the controller falls back to two passive behaviors driven by the vision module:

```
1 else:
2     if need_scan:
3         # Rotate in place to sweep the camera FOV
4         drive_chassis(ep_chassis, forward_speed=0, turn_speed=30)
5     else:
6         chassis_stop(ep_chassis)
```

The `need_scan` flag is triggered by the finite state machine's `SEARCHING` state, indicating that coverage path following has been temporarily suspended, pending local re-detection. The in-place rotation at a fixed 30 RPM sweeps the 120° camera field of view across the surrounding area; once a new target is detected, the chasing sequence automatically re-engages.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

5 USER INTERFACE DESIGN

This section describes the user interface design of the CourtSweeper iOS application. Developed natively in Swift using the SwiftUI framework, the mobile application serves as the primary remote command center and implements the **iOS Interrupt Control Layer** of the hierarchical FSM (Section 3.7). It connects directly to the Jetson Orin NX's localized Wi-Fi hotspot, providing real-time situational awareness, manual teleoperation capabilities, high-priority interrupt commands, and system telemetry monitoring.

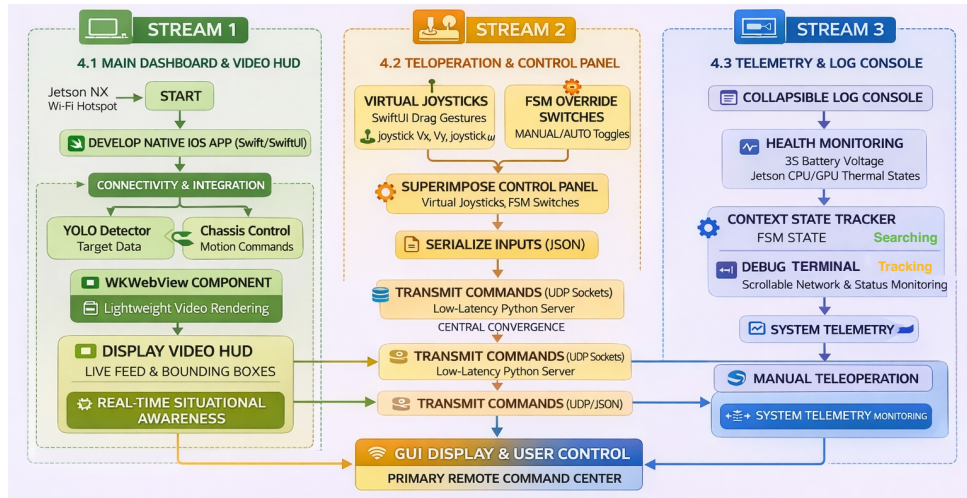


Figure 13: The software architecture and data flow for the CourtSweeper iOS application, illustrating the centralized SwiftUI core connected via bidirectional Wi-Fi and UDP sockets to external Jetson computing units, and featuring functional breakdowns of main dashboard, teleoperation, and telemetry modules.

5.1 Main Dashboard & Video HUD

The primary interface is designed as a landscape-oriented Heads-Up Display (HUD) to maximize the operator's field of view. The core component of this view is the video presentation layer.

- **Video Streaming Rendering:** Instead of relying on complex and latency-prone H.264 decoders on the client side, the app utilizes a lightweight WKWebView component to render the MJPEG video stream broadcasted by the Jetson unit.
- **AR Overlay:** The AI-processed bounding boxes, target crosshairs, and ball coordinates are overlaid directly onto the video feed, providing the operator with intuitive, real-time feedback on the YOLO perception module's performance.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

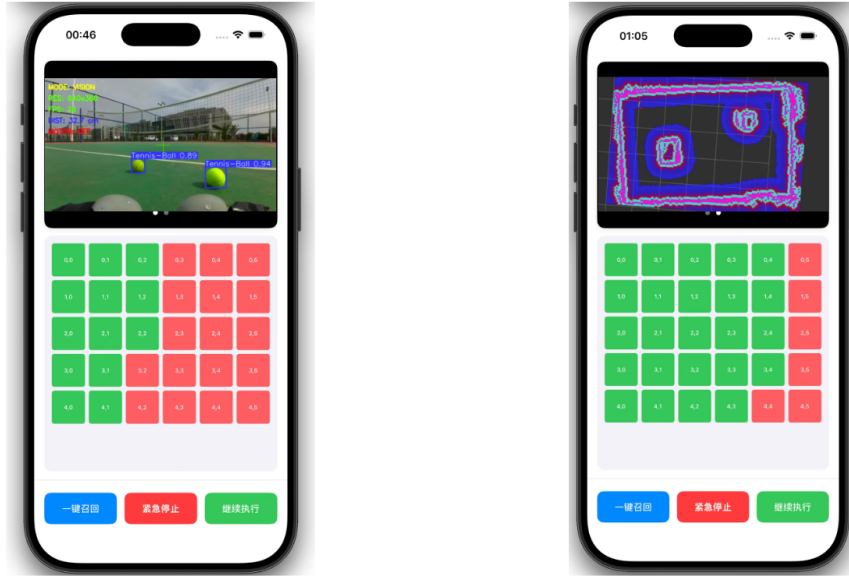


Figure 14: User interface layout of the CourtSweeper iOS application. The primary HUD renders the low-latency MJPEG video stream overlaid with real-time YOLO bounding boxes. The lower panel provides virtual joysticks for holonomic teleoperation and manual override switches for the system’s Finite State Machine (FSM).

5.2 Teleoperation & Control Panel

Superimposed on the lower sections of the HUD is the Control Panel, designed for ergonomic thumb access on an iPhone or iPad.

- **Virtual Joysticks:** Implemented via SwiftUI drag gestures, the left joystick dictates the omnidirectional translational movement (V_x, V_y) of the Mecanum chassis, while the right joystick controls rotational velocity (ω).
- **FSM Override Switches:** A prominent set of toggle buttons implements the upper-layer FSM interrupt commands described in Section 3.7. The E-STOP button transmits an emergency stop command that immediately engages chassis braking and suspends all autonomous logic. The RECALL button triggers a one-touch recall, commanding Nav2 to navigate the robot back to the origin pose. The AUTO / MANUAL toggle switches the robot between autonomous mission execution and manual teleoperation mode.
- **Command Transmission:** Control inputs are serialized into lightweight JSON packets and transmitted to the Jetson’s Python server via low-latency UDP sockets using Apple’s native Network framework.

5.3 Telemetry & Log Console

To facilitate debugging and monitor system health during field tests, a collapsible Log Console is integrated into the interface.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

- **Health Monitoring:** Displays critical hardware telemetry parsed from the DJI SDK, including the 3S battery voltage (preventing over-discharge) and the Jetson's CPU/GPU thermal states.
- **State Machine Tracker:** A dynamic label continuously reflects the robot's current FSM state from the lower-layer autonomous workflow (e.g., *MAPPING* → *NAVIGATING* → *INTERCEPTING* → *RETRIEVING* → *SCANNING*), as well as any active upper-layer interrupt state (*E-STOP* or *RECALLING*).
- **Debug Terminal:** A scrolling text view (implemented via `ScrollView` in SwiftUI) outputs real-time network connection statuses, ping latency, and critical error logs thrown by the Python backend.

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

6 THEORETICAL MANUAL

This section provides an overview of the mathematical and algorithmic foundations underpinning each subsystem of the CourtSweeper. For the complete formal derivations, parameter tables, and mathematical proofs, the reader is referred to the standalone **CourtSweeper Theoretical Manual** (TM/theory_manual.pdf), which presents these topics in full detail. The subsections below summarise the scope of each theoretical domain and indicate where the corresponding design and implementation are described in the preceding chapters.

6.1 Perception Models

The perception subsystem is built on the standard pinhole camera model (intrinsic matrix K and projection matrix P), Image-Based Visual Servoing (IBVS) for direct image-plane error computation, a nearest-target depth proxy via the bottom-y heuristic, and the YOLOv6-Nano single-stage object detector (comprising EfficientRep backbone, Rep-PAN neck, and decoupled head). The LiDAR sensor operates on the laser triangulation principle, while the IR proximity sensor provides continuous range data. The camera model underpins the vision pipeline in Section 4.1; the LiDAR and IR sensors are integrated in the hardware design (Section 2) and the reactive control loop (Section 4).

6.2 Mecanum Wheel Kinematics

The CourtSweeper employs two formulations of the Mecanum inverse kinematics: a full 3-DOF holonomic model (v_x, v_y, ω_z) for Nav2-based autonomous navigation, and a reduced 2-DOF model (v_f, v_t) for reactive visual servoing. Both decompose platform-level velocity commands into per-wheel RPM targets. These equations are implemented in the chassis bridge (Section 3) and the reactive drive module (Section 4).

6.3 PID Control Theory

The reactive chassis controller employs two independent PID regulators in the standard parallel form. The lateral alignment controller (pid_turn) drives the image-plane centroid offset to zero; the longitudinal approach controller (pid_forward) regulates the IR-measured distance toward the ball-contact setpoint. The two controllers operate in a priority-ordered cascade that includes a blind-zone relay and a sliding-window ingestion confirmation detector. Their implementation and gain schedules are provided in Section 4.

6.4 Navigation and Planning Theory

Autonomous navigation rests on three pillars: pose-graph SLAM via slam_toolbox (online asynchronous mode with Karto-style scan matching and non-linear least-squares optimisation), Adaptive Monte Carlo Localisation (AMCL) via a particle filter with KLD-sampling, and Nav2's Smac Hybrid-A* global planner combined with the Regulated Pure Pursuit local controller. The

Vision-Based Retriever System (Short for VBRS)	Issue: 1.0
Hardware/Software Design Description (HSDD)	Issue Date: 03 / 23 / 2026

coverage strategy follows a Boustrophedon S-shaped pattern over a discretised occupancy grid. These algorithms are deployed in the SLAM and Nav2 pipelines described in Section 3.

6.5 Finite State Machine

The behavioural logic is formally defined as a Mealy-type FSM $\mathcal{M} = (Q, \Sigma, \delta, q_0, F)$ with four states (IDLE, SEARCHING, TRACKING, INGESTING), nine input events spanning perception, sensor, timer, and UI triggers, and a transition function that governs both the autonomous mission workflow and interrupt handling. The hierarchical two-layer FSM architecture is detailed in Section 3.7, and the phase-3 reactive control implementation is presented in Section 4.

6.6 Coordinate Transformations

The ROS2 TF tree defines rigid transformations among the odom, base_footprint, base_link, and laser frames. Euler-to-quaternion conversion and yaw normalisation prevent numerical drift; a coordinate sign convention distinguishes mapping mode (negated x and yaw) from navigation mode (raw DJI coordinates). These transforms are implemented in the mapping and navigation bridges (Section 3).

6.7 Communication Protocol Specifications

Inter-process communication follows three distinct protocols: ROS2 topics (/odom, /scan, /cmd_vel, /camera/image/compressed, /camera/camera_info, /tf) bridging the DJI chassis with the SLAM and vision stacks; a UDP-based JSON command protocol from the iOS app to the Jetson; and a serial CSV protocol (115200 bps, 8N1) from the Jetson to the Arduino motor driver. The ROS2 bridge and Wi-Fi modes are specified in Section 3; the Arduino firmware is described in Section 4.

6.8 Motor Drive Theory

The dual-roller intake motors are driven by 8-bit PWM signals generated by the Arduino firmware, controlling two BTS7960 full H-bridge modules. A firmware-level communication watchdog halts all motors within 500 ms of serial link loss. The PWM duty-cycle derivation, BTS7960 truth table, and command-parsing logic are presented in the Theoretical Manual; the corresponding implementation is in Section 4.